Battery Vision Localization and Pick-and-Place Flow

Meituan Challenge Robot Code Guide 02 | English version | 2026-10-13

1 Overview

The `Battery` module in `battery.docx` implements vision-guided battery picking and placing. It first attempts to obtain three coordinate pairs for blue, green, and yellow targets. When the multiple-target condition is met, it picks and places them in the order yellow, green, then blue; otherwise, it requests a red target and executes the single-battery task. The module connects main-program scheduling, VisionMaster results, and robot motion, and contains the main task logic in the supplied code.

Task 1 and Task 2 are labels for the current project's single-red-target and three-color-target flows. They do not represent the full battery-swapping operation for the entire competition. The current code contains no confirmable logic for electrical connection detection, mechanical locking confirmation, or acceptance checks for a completed battery swap.

2 Procedure Responsibilities

Procedure	Inputs and dependencies	Main behavior
Battery_get_values	input_string, camera reference pose, communication module	Request vision results, check the results, and add position offsets
Battery_work	Six parsed values, reply string	Check for multiple targets first, then fall back to a red target
Battery_work_1	nval1, nval2, red placement target	Pick and place a single red battery
Battery_work_2	Three coordinate pairs, three color-specific placement targets	Pick and place yellow, green, and blue batteries in sequence

3 Coordinate Acquisition and Task Selection

`Battery_get_values` first moves to `Redy_Camara_Diamonds` and waits for the robot to reach the target. It copies `input_string` to `send_string`, sets `socket_start = TRUE` and `startPLC = FALSE`, waits for 0.1 seconds, then calls `Connect2`. The communication module parses the vision reply into `nval1` through `nval6`.

If both of the first two values are zero, or the reply string equals `",;"`, the procedure prints an error or no-target message and returns. Otherwise, it adds `diamondX` and `diamondY` to the respective X/Y values in all three pairs, then moves to the camera reference pose. This `RETURN` exits only the current procedure; it does not return a separate Boolean success value to the caller.

The current execution sequence in `Battery_work` is:

1. Set `input_string = "Getvalues"` and obtain coordinates.

2. Enter `Battery_work_2` if the fifth and sixth values do not both equal their respective common offset values, the first pair is not both zero, and the reply is not `",;"`. Exit the current task iteration after that procedure completes.

3. If this condition is not met, switch to `"Redvalues"`, obtain coordinates again, and check for no target.

4. Call `Battery_work_1` when red-target coordinates are detected. The current code then makes a second `"Redvalues"` request and check; if a target is still detected, it calls `Battery_work_1` again.

The current single-red-target branch therefore does not always perform exactly one pick per iteration. The second request is not a dedicated error-retry branch. This guide preserves that behavior; the team needs to check whether it matches the intended on-site task.

4 Coordinate Mapping and Pick-and-Place Order

Branch	Coordinate fields	Execution order	Placement target actually used by MoveL
Single red target	nval1, nval2	Red	D_put_Red
Three-color targets	nval5, nval6	First: yellow	D_put_Yellow
Three-color targets	nval3, nval4	Second: green	D_put_Green
Three-color targets	nval1, nval2	Third: blue	D_put_Blue

The three-color data fields are organized as blue, green, then yellow, while execution proceeds as yellow, green, then blue. Some comments and log messages in the yellow and blue sections state that the battery is placed at the other color's location, but the actual motion instructions use `D_put_Yellow` and `D_put_Blue`, respectively. The executable statements shown in the table take precedence; their physical station meanings must be checked against the on-site taught targets.

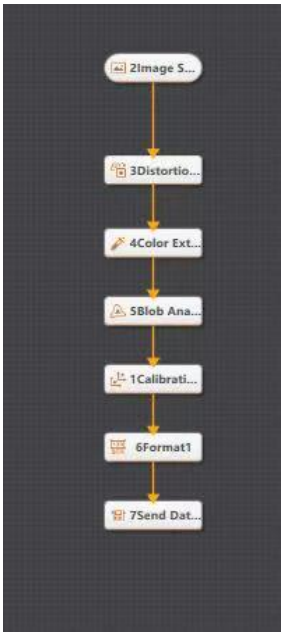
Each valid target follows a similar pick-and-place sequence: calculate offsets using the vision X/Y values, camera reference pose, and empirical color-specific offsets; descend in stages from the approach position; issue the gripping output with `Set IIO_do2`; lift and move to the placement target; release with `ReSet IIO_do2`; then leave the placement area. The three-color branch returns to the camera reference pose after processing all targets, and the single-color branch also returns there after placement.

The code uses `MoveL`, `RelTool`, `Offs`, `WaitRob`, and `WaitTime` to organize these actions. `offSetAngle` remains fixed at zero throughout the current pick-and-place procedures, so the third parsed value cannot be described as already participating in orientation adjustment. The material does not specify the mechanical device actuated by the gripping output or its feedback detection.

5 VisionMaster Flows

5.1 Task 1: Single-Target Flow

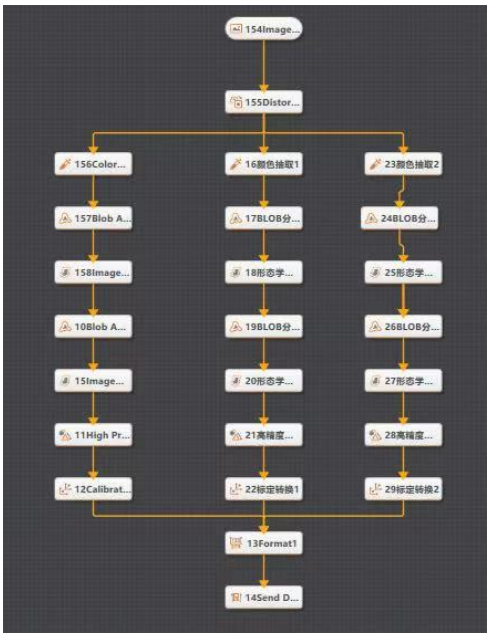
The serial nodes identifiable in the image include image source, distortion processing, color extraction, Blob analysis, calibration, formatting, and data transmission. The robot's single-red-target branch uses the `Redvalues` request. Trigger bindings, color thresholds, Blob filtering parameters, and output templates need to be checked in the original VisionMaster project.



Original VisionMaster Task 1 flow diagram

5.2 Task 2: Multiple-Target Flow

After image and distortion processing, the diagram branches into three color-processing paths. Each path contains nodes for Blob, image or morphological processing, and calibration or coordinate conversion, before merging into `Format1` and a data-transmission node. The robot's `Getvalues` request corresponds to the code path that consumes three pairs of values.



Original VisionMaster Task 2 flow diagram

These two images are flow screenshots and do not contain a complete, readable configuration. Some labels are truncated. The screenshots alone cannot confirm each vision branch's exact color binding, calibration matrix, output units, or algorithm parameters.

6 Limitations and Checks

- **Validity checks are incomplete.** The results of `StrToVal` conversions do not form a unified success flag for the calling logic. Coordinate offsets are applied after some no-target checks, and checking zero values after compensation cannot independently prove that the original target was valid.

- **The three-color targets share a single coordinate acquisition.** The current procedure does not capture a new image before each pick. Handling target movement during execution requires on-site assessment.

- **Network errors and stale values.** Parsing resets all six numeric values, but whether stale values can still enter the current iteration's logic when a communication error occurs before parsing needs to be verified.

- **An output does not confirm a successful pick.** Setting a digital output and waiting for a fixed time do not prove that a workpiece has been gripped or placed securely. No corresponding sensor confirmation was found in the material.

- **Motion parameters depend on the workstation.** The original calibrated coordinates, color-specific offsets, and motion parameters are not general-purpose configurations that can be run directly.

These points define the limits of this guide; they do not indicate that the code has been modified as part of this work. The program has not been compiled, simulated, or executed on a physical robot during this work.

7 Source References

Source version: `lkkkkk667/meituan`, commit `68e262bbedcf2269ba3a7321a045f97e96f185b6`.

- `battery.docx`: `Battery_get_values`, `Battery_work`, `Battery_work_1`, and `Battery_work_2`.

- `connect_socket.docx`: parsing and resetting the six fields in `ProcessReceivedData`.

- `Modul.docx`: shared variables, camera reference pose, and color-specific placement targets.

- `task1.jpg`, `task2.jpg`: original VisionMaster flow screenshots.

Related guides: Main Program · Communication Module.